# O Filtro de Kalman: Teoria e Prática

Linear e Unscented

Júnior Bandeira

Fevereiro de 2026

## Índice

**1 Introdução ao Filtro de Kalman** **2**

**2 O Filtro de Kalman Linear (LKF)** **2**
2.1 As Variáveis Fundamentais . . . . . 2
2.2 As Equações Matemáticas . . . . . 2
2.3 Exemplo Prático LKF: Rastreio de Altitude 1D . . . . . 3

**3 O Unscented Kalman Filter (UKF)** **4**
3.1 A Solução Unscented (A Transformada) . . . . . 4
3.2 As Etapas Matemáticas do UKF . . . . . 4

**4 Implementação Prática em C (LKF 1D)** **5**
4.1 A Estrutura de Dados . . . . . 5
4.2 A Inicialização (O Chute Inicial) . . . . . 5
4.3 O Loop Principal (Predição e Atualização) . . . . . 6

**5 Implementação Prática em C (UKF 1D)** **7**
5.1 A Estrutura de Dados e Sintonia (Tuning) . . . . . 7
5.2 A Fatoração de Cholesky . . . . . 8
5.3 O Ciclo de Atualização (Transformada Unscented) . . . . . 9
5.4 Nota de Engenharia: A Armadilha do Excesso de Processamento . . . . . 11

# 1 Introdução ao Filtro de Kalman

Sensores físicos possuem ruído. Modelos matemáticos possuem incertezas. O Filtro de Kalman é um algoritmo ótimo que funde essas duas fontes de informação imperfeitas para estimar o estado real de um sistema dinâmico de forma incrivelmente precisa.

Em essência, o filtro funciona em um ciclo perpétuo de duas etapas:

1. **Predição (Predict):** Usa as leis da física e o estado anterior para adivinhar onde o sistema deveria estar agora.
2. **Atualização (Update):** Lê o sensor e usa essa medida para corrigir a adivinhação, pesando qual das duas fontes é mais confiável (o sensor ou a física).

# 2 O Filtro de Kalman Linear (LKF)

O LKF puro assume que o sistema evolui de forma estritamente linear e que os ruídos seguem uma distribuição Gaussiana normal.

## 2.1 As Variáveis Fundamentais

Para entender as equações, precisamos definir o que estamos calculando:

- $x$: O **Vetor de Estado** (ex: posição e velocidade atual).
- $P$: A **Matriz de Covariância** (nossa incerteza sobre o estado $x$).
- $F$: A **Matriz de Transição de Estado** (a física do sistema, como a posição muda com a velocidade ao longo do tempo).
- $H$: A **Matriz de Observação** (mapeia o que queremos saber para o que o sensor realmente consegue ler).
- $Q$: O **Ruído de Processo** (a incerteza do nosso modelo físico, ex: rajadas de vento não calculadas).
- $R$: O **Ruído de Medição** (a variação estática do sensor, lida no datasheet).
- $K$: O **Ganho de Kalman** (o fator de confiança: de 0 a 1, quem merece mais confiança, o sensor ou o modelo?).

## 2.2 As Equações Matemáticas

**Passo 1: Predição (A Expectativa)**
Onde achamos que estamos, baseado no passado?

$$x_{k|k-1} = Fx_{k-1|k-1} + Bu_k$$

Qual a nossa incerteza agora? (A incerteza sempre cresce na predição).

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

**Passo 2: Atualização (A Realidade)**
O sensor nos dá uma leitura $z_k$. Calculamos o resíduo $y_k$ (a diferença entre o que o sensor leu e o que previmos):

$$y_k = z_k - Hx_{k|k-1}$$

Calculamos a covariância da inovação $S_k$ e, a partir dela, o Ganho de Kalman $K_k$:

$$S_k = HP_{k|k-1}H^T + R$$

$$K_k = P_{k|k-1}H^T S_k^{-1}$$

Finalmente, atualizamos nosso estado e diminuímos nossa incerteza:

$$x_{k|k} = x_{k|k-1} + K_k y_k$$

$$P_{k|k} = (I - K_k H)P_{k|k-1}$$

## 2.3 Exemplo Prático LKF: Rastreio de Altitude 1D

Imagine um foguete ou drone subindo. Queremos estimar sua Altitude ($p$) e Velocidade ($v$). O estado é $x = \begin{bmatrix} p \\ v \end{bmatrix}$.

Se o nosso laço roda a cada $\Delta t$ segundos, a física básica (cinemática) diz que $p_{novo} = p_{velho} + v_{velho}\Delta t$. A velocidade se mantém (ignorando aceleração por simplicidade). Nossa Matriz $F$ fica:

$$F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$$

Nosso sensor é um barômetro. Ele **só lê altitude**, não lê velocidade. A matriz $H$ extrai apenas a primeira linha do estado:

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Se o barômetro tem um ruído conhecido de $\pm 2$ metros, a variância do sensor é $2^2 = 4$. Então $R = \begin{bmatrix} 4 \end{bmatrix}$.

Durante o voo, o filtro faz o seguinte:

1. Multiplica o estado anterior pela matriz $F$. Ele prevê: "Pela velocidade que eu estava, devo estar em 100m".

2. O barômetro entrega $z_k = 105m$. A diferença ($y_k$) é $5m$.

3. O Ganho de Kalman $K$ avalia a matriz $P$ (nossa incerteza teórica) contra $R$ (ruído de 4). Se a velocidade estava flutuando muito, $K$ será alto (ex: 0.8), e o filtro puxa o estado para perto de 105m. Se a nossa predição estava muito sólida, $K$ será baixo (ex: 0.2), e o filtro ignora parte dos 105m, assumindo que foi ruído atmosférico.

# 3 O Unscented Kalman Filter (UKF)

O LKF é perfeito, **desde que** as matrizes $F$ e $H$ sejam lineares. Mas o mundo real é não-linear. Se a dinâmica do sistema for um pêndulo (envolve seno e cosseno), ou se envolver arrasto aerodinâmico (velocidade ao quadrado), o sistema quebra a suposição do LKF.

Se você passar uma distribuição Gaussiana (uma curva em sino perfeita) por uma equação não-linear complexa, ela sai deformada do outro lado. Ela deixa de ser Gaussiana, e o filtro tradicional diverge.

Uma solução histórica é o *Extended Kalman Filter* (EKF), que usa cálculo diferencial (Jacobianas) para forçar a não-linearidade a virar uma reta tangente temporária. O problema é que calcular derivadas parciais pesadas no silício de um microcontrolador é lento e matematicamente custoso, e a aproximação linear gera muitos erros em curvas agudas.

## 3.1 A Solução Unscented (A Transformada)

A genialidade do UKF reside na sua premissa central: *"É mais fácil aproximar uma distribuição de probabilidade do que aproximar uma função não-linear arbitrária"*.

Em vez de calcular derivadas (Jacobianas), o UKF usa a **Transformada Unscented (UT)**:

1. **Geração dos Pontos Sigma:** O algoritmo seleciona um pequeno e simétrico conjunto de pontos exatos ao redor da nossa estimativa atual (a média). Se o estado tem dimensão $L$ (ex: 2 para posição e velocidade), o UKF escolhe $2L + 1$ pontos, chamados de Pontos Sigma.

2. **Propagação Não-Linear:** Em vez de passar a matriz de incerteza inteira pela função $F$, nós simplesmente calculamos a física (a equação real com arrasto, senos e cossenos) de forma independente para **cada um** dos Pontos Sigma.

3. **Reconstrução Gaussiana:** Com os pontos transformados no "futuro", o UKF usa uma soma ponderada (pesos estatísticos predefinidos) para calcular onde está o novo centro (média) e qual é a nova "nuvem de dispersão" (matriz de covariância $P$).

## 3.2 As Etapas Matemáticas do UKF

As equações do UKF exigem operações de álgebra linear mais densas (como Fatoração de Cholesky para gerar os Pontos Sigma), mas a lógica do ciclo se mantém:

1. **Calcular Pontos Sigma:**

$$X = \begin{bmatrix} x & x + \gamma\sqrt{P} & x - \gamma\sqrt{P} \end{bmatrix}$$

2. **Predição (Propagação):** Passamos os pontos pela equação física bruta:

$$X_{k|k-1} = f(X_{k-1})$$

3. **Média e Covariância Prevista:**

$$x_{k|k-1} = \sum W_i^m X_i$$

$$P_{k|k-1} = \sum W_i^c (X_i - x)(X_i - x)^T + Q$$

4. **Atualização (Medição):** Passamos os pontos previstos pela função não-linear do sensor $Z = h(X)$, calculamos a média das medições esperadas, cruzamos a covariância $P_{xz}$ e encontramos o Ganho de Kalman de forma análoga ao LKF.

**O Veredito do UKF:** Ele atinge precisão de até 3ª ordem na série de Taylor para qualquer função não-linear, sem exigir uma única linha de código calculando derivadas.

# 4 Implementação Prática em C (LKF 1D)

Na teoria, o Filtro de Kalman é baseado em álgebra matricial. Na prática de sistemas embarcados em linguagem C, multiplicar matrizes dinamicamente consome muitos ciclos de CPU. A técnica de engenharia correta é "desdobrar" as matrizes: resolvemos as multiplicações manualmente no papel (para o nosso caso específico de 2 dimensões) e transcrevemos apenas as equações resultantes para o código.

Abaixo, dissecamos a implementação de um LKF para rastreio de altitude e velocidade, com base no código de voo de um altímetro embarcado.

## 4.1 A Estrutura de Dados

Agrupamos o estado atual e as matrizes de incerteza em uma única `struct`. Isso mantém a memória organizada e permite instanciar múltiplos filtros simultâneos (por exemplo, um para atitude e outro para altitude).

```c
// Estrutura do Filtro Kalman Linear (LKF)
typedef struct {
    float x[2];       // Estado: [0]=Altitude, [1]=Velocidade
    float P[2][2];    // Matriz de Covariancia (Incerteza atual)
    float Q[2][2];    // Ruido do Processo (Incerteza do modelo fisico)
    float R;          // Ruido da Medida (Incerteza fixa do Sensor Barometrico)
} LKF_t;
```

## 4.2 A Inicialização (O Chute Inicial)

Antes do voo (ou no momento da decolagem), o filtro precisa de um ponto de partida. Configurar o `Q` e o `R` é o que chamamos de "Sintonia" (*Tuning*) do filtro.

```c
void LKF_Init(LKF_t* f, float alt_inicial) {
    // Estado Inicial: Conhecemos a altitude, e a velocidade e zero no chao
    f->x[0] = alt_inicial;
    f->x[1] = 0.0f;

    // Covariancia Inicial (P): Qual nossa confianca no estado acima?
    // 1.0 significa incerteza padrao. Sem correlacao cruzada inicial (0.0).
```

```
    f->P[0][0] = 1.0f; f->P[0][1] = 0.0f;
    f->P[1][0] = 0.0f; f->P[1][1] = 1.0f;

    // Ruido de Processo (Q): A fisica falha. Nosso modelo assume velocidade
    // constante, mas um foguete sofre aceleracao brutal. O Q "avisa" o filtro
    // que o modelo fisico tem incertezas, abrindo espaco para confiar no sensor.
    f->Q[0][0] = 0.01f; f->Q[0][1] = 0.0f;
    f->Q[1][0] = 0.0f;  f->Q[1][1] = 0.1f;

    // Ruido de Medida (R): A variancia natural do MS5611 ou BMP280.
    f->R = 0.20f;
}
```

## 4.3 O Loop Principal (Predição e Atualização)

A função de `Update` é chamada toda vez que o sensor gera um dado novo. Note como as equações matriciais pesadas se transformam em somas e multiplicações flutuantes ultra-rápidas.

```
float LKF_Update(LKF_t* f, float z_meas, float dt, float* vel_out) {

    /* ========================================================
       ETAPA 1: PREDICAO (Onde a fisica diz que estamos)
       ======================================================== */
    float x_pred[2];

    // Equacao: X_pred = F * X
    // Cinemática basica: Nova Altitude = Alt Antiga + (Velocidade * DeltaTempo)
    x_pred[0] = f->x[0] + f->x[1] * dt;
    x_pred[1] = f->x[1]; // No modelo simples, velocidade se mantem

    float P_pred[2][2];
    // Equacao: P_pred = F * P * F^T + Q
    // Expansao algebrica da multiplicacao das matrizes 2x2:
    P_pred[0][0] = f->P[0][0] + dt * (f->P[1][0] + f->P[0][1]) + dt * dt * f->P[1][1] + f->Q[0][0];
    P_pred[0][1] = f->P[0][1] + dt * f->P[1][1] + f->Q[0][1];
    P_pred[1][0] = f->P[1][0] + dt * f->P[1][1] + f->Q[1][0];
    P_pred[1][1] = f->P[1][1] + f->Q[1][1];

    /* ========================================================
       ETAPA 2: ATUALIZACAO (Correcao com a realidade)
       ======================================================== */
    float K[2];

    // Equacao: S = H * P_pred * H^T + R
    // Como H = [1 0] (porque o sensor so le altitude, nao velocidade),
    // a matematica colapsa lindamente para uma unica soma:
    float S = P_pred[0][0] + f->R;

    // Equacao do Ganho: K = P_pred * H^T * inv(S)
    K[0] = P_pred[0][0] / S;
    K[1] = P_pred[1][0] / S;
```

```c

    // Atualiza o Estado: X_novo = X_pred + K * (Medida - X_pred)
    // O termo (z_meas - x_pred[0]) e a "Surpresa" do sensor (Inovacao)
    f->x[0] = x_pred[0] + K[0] * (z_meas - x_pred[0]);
    f->x[1] = x_pred[1] + K[1] * (z_meas - x_pred[0]);

    // Atualiza a Covariancia: P_novo = (I - K * H) * P_pred
    float P_new[2][2];
    P_new[0][0] = (1.0f - K[0]) * P_pred[0][0];
    P_new[0][1] = (1.0f - K[0]) * P_pred[0][1];
    P_new[1][0] = -K[1] * P_pred[0][0] + P_pred[1][0];
    P_new[1][1] = -K[1] * P_pred[0][1] + P_pred[1][1];

    // Copia a nova matriz de volta para a memoria persistente da struct
    memcpy(f->P, P_new, sizeof(P_new));

    // Exporta a velocidade descoberta indiretamente (Opcional)
    if (vel_out) *vel_out = f->x[1];

    return f->x[0]; // Retorna a altitude limpa e filtrada
}
```

**A Verdadeira Magia da Covariância Cruzada:** Observe atentamente a linha `f->x[1] = x_pred[1] + K[1] * (z_meas - x_pred[0]);`. O nosso barômetro é estúpido; ele nunca leu velocidade, apenas pressão atmosférioca. No entanto, o filtro de Kalman pega o erro posicional (*o quanto erramos na altitude*) e o multiplica pelo ganho de velocidade (`K[1]`). Através do comportamento da física atrelada ao tempo, o filtro consegue **adivinhar a velocidade real** do foguete apenas observando o quão rápido a altitude está falhando nas predições!

# 5 Implementação Prática em C (UKF 1D)

Embora a implementação orientada a objetos (C++) seja comum no ecossistema Arduino, o padrão ouro na indústria de sistemas embarcados críticos (como no STM32) é escrever os filtros puramente em C. Usamos `structs` e matrizes estáticas para garantir previsibilidade extrema no uso de memória e evitar alocações dinâmicas.

Abaixo, traduzimos a lógica do Unscented Kalman Filter para C, baseada em um modelo cinemático otimizado e recheada com a intuição física de cada variável.

## 5.1 A Estrutura de Dados e Sintonia (Tuning)

O UKF exige não apenas as matrizes de ruído, mas também os parâmetros da Transformada Unscented ($\alpha$, $\beta$, $\kappa$) e os pesos estatísticos.

```c
#include <math.h>

// Estrutura do Filtro Unscented Kalman (UKF)
typedef struct {
    float estado[2];       // Estado: [0]=Altitude, [1]=Velocidade
    float P[2][2];         // Matriz de Covariancia do Erro
    float Q[2][2];         // Ruido do Processo
    float R;               // Ruido da Medida (Sensor)

    // Parametros da Transformada Unscented
```

```c
    float lambda;          // Fator de escala composto
    float pesos[3];        // Pesos dos pontos sigma (versao otimizada de 3 pontos)
} UKF_t;

void UKF_Init(UKF_t* f, float alt_inicial) {
    // ESTADO INICIAL [x]
    f->estado[0] = alt_inicial; // Ponto de chute inicial
    f->estado[1] = 0.0f;        // Velocidade inicial

    // COVARIANCIA INICIAL [P] - "O quanto eu confio no meu chute inicial?"
    // Matriz Identidade -> Incerteza padrao, sem correlacao entre posicao e velocidade.
    f->P[0][0] = 1.0f; f->P[0][1] = 0.0f;
    f->P[1][0] = 0.0f; f->P[1][1] = 1.0f;

    // RUIDO DO PROCESSO [Q] - "O quanto a fisica do mundo real e caotica?"
    // Valores baixos = confia muito na lei da inercia.
    // Valores altos = o foguete treme muito ou sofre forças externas nao modeladas.
    f->Q[0][0] = 0.001f; f->Q[0][1] = 0.0f;
    f->Q[1][0] = 0.0f;   f->Q[1][1] = 0.001f;

    // RUIDO DA MEDIDA [R] - "O quanto o sensor BMP280 e ruim?"
    f->R = 0.1f;

    // PARAMETROS DO UKF (Unscented Transform)
    // Esses valores definem o quao longe os "Pontos Sigma" (batedores) vao ser jogados.
    float alpha = 1.0f, kappa = 0.0f;
    f->lambda = alpha * alpha * (2.0f + kappa) - 2.0f;

    // PESOS [W] - A media ponderada dos pontos sigma
    // O ponto central (media) tem um peso diferente dos pontos perifericos.
    f->pesos[0] = f->lambda / (2.0f + f->lambda);
    f->pesos[1] = 1.0f / (2.0f * (2.0f + f->lambda));
    f->pesos[2] = f->pesos[1];
}
```

## 5.2 A Fatoração de Cholesky

Para gerar a "nuvem" de Pontos Sigma, precisamos extrair a raiz quadrada da matriz de covariância. Em álgebra linear, isso é feito pela Decomposição de Cholesky, que transforma nossa matriz em uma matriz triangular inferior.

```c
// Recebe matriz A (entrada) e matriz L (saida)
void UKF_Cholesky(float A[2][2], float L[2][2]) {
    // L e uma matriz triangular inferior tal que L * L_transposta = A
    // E o equivalente matricial de tirar a raiz quadrada de um numero.
    L[0][0] = sqrtf(A[0][0]);
    L[0][1] = 0.0f; // Triangular inferior (acima da diagonal e zero)

    // Garante estabilidade evitando divisao por zero
    if (L[0][0] > 0.0001f) {
```

```
        L[1][0] = A[1][0] / L[0][0];
    } else {
        L[1][0] = 0.0f;
    }

    // Protecao contra instabilidade numerica (raiz de negativo)
    float val_interno = A[1][1] - (L[1][0] * L[1][0]);
    if (val_interno < 0.0f) val_interno = 0.0f;

    L[1][1] = sqrtf(val_interno);
}
```

## 5.3 O Ciclo de Atualização (Transformada Unscented)

No ciclo principal, espalhamos os pontos sigma, aplicamos a física a cada um deles, e reconstruímos a estatística final.

```
float UKF_Update(UKF_t* f, float z_meas, float dt, float* vel_out) {

    /* ========================================================
       1. GERAR PONTOS SIGMA ("Os Batedores")
       ======================================================== */
    float L[2][2];
    UKF_Cholesky(f->P, L); // L e o nosso desvio medio

    float pts[2][3]; // 3 pontos para 2 dimensoes (1 media + 2 desvios)
    float fator = sqrtf(2.0f + f->lambda); // Distancia estatistica dos
   pontos

    // Ponto 1: Onde achamos que estamos (Media)
    pts[0][0] = f->estado[0];
    pts[1][0] = f->estado[1];

    // Ponto 2: Um pouco "pra cima/rapido" (Media + Desvio)
    pts[0][1] = f->estado[0] + fator * L[0][0];
    pts[1][1] = f->estado[1] + fator * L[1][0];

    // Ponto 3: Um pouco "pra baixo/lento" (Media - Desvio)
    pts[0][2] = f->estado[0] - fator * L[0][0];
    pts[1][2] = f->estado[1] - fator * L[1][0];

    /* ========================================================
       2. PREDICAO / PROPAGACAO FISICA
       ======================================================== */
    // Empurra os 3 pontos para o futuro usando Fisica Newtoniana
    float pred_pts[2][3];
    for (int i = 0; i < 3; i++) {
        // Nova Posicao = Posicao Antiga + Velocidade * Tempo
        pred_pts[0][i] = pts[0][i] + pts[1][i] * dt;
        // Nova Velocidade = Velocidade Antiga (Modelo Constante)
        pred_pts[1][i] = pts[1][i];
    }

    /* ========================================================
       3. RECONSTRUIR MEDIA E COVARIANCIA PREDITAS
       ======================================================== */
    // Calcula onde a nuvem de pontos foi parar em media
```

```c
    float x_pred[2] = {0.0f, 0.0f};
    for (int i = 0; i < 3; i++) {
        x_pred[0] += f->pesos[i] * pred_pts[0][i];
        x_pred[1] += f->pesos[i] * pred_pts[1][i];
    }

    // Covariancia do novo estado predito em relacao aos pontos sigma
    float P_pred[2][2] = {{0.0f, 0.0f}, {0.0f, 0.0f}};
    for (int i = 0; i < 3; i++) {
        float dx = pred_pts[0][i] - x_pred[0];
        float dv = pred_pts[1][i] - x_pred[1];

        P_pred[0][0] += f->pesos[i] * dx * dx;
        P_pred[0][1] += f->pesos[i] * dx * dv;
        P_pred[1][1] += f->pesos[i] * dv * dv;
    }
    P_pred[1][0] = P_pred[0][1]; // Simetriza a matriz

    // Adiciona o Ruido do Processo [Q]
    P_pred[0][0] += f->Q[0][0];
    P_pred[1][1] += f->Q[1][1];

    /* ========================================================
       4. ATUALIZACAO COM O SENSOR
       ======================================================== */
    // O que esperamos ler no sensor? (Neste caso simples, a propria altitude)
    float z_pred = x_pred[0];
    float P_zz = 0.0f; // Covariancia Medicao
    float P_xz[2] = {0.0f, 0.0f}; // Covariancia Cruzada

    for(int i = 0; i < 3; i++) {
        float dz = pred_pts[0][i] - z_pred;
        P_zz += f->pesos[i] * dz * dz;
        P_xz[0] += f->pesos[i] * (pred_pts[0][i] - x_pred[0]) * dz;
        P_xz[1] += f->pesos[i] * (pred_pts[1][i] - x_pred[1]) * dz;
    }
    P_zz += f->R; // Adiciona o ruido do sensor

    // Ganho de Kalman [K]
    float inv_S = 1.0f / P_zz; // Divisao e lenta, fazemos 1 vez
    float K[2] = {P_xz[0] * inv_S, P_xz[1] * inv_S};

    // APLICAR A CORRECAO FINAL
    // Residuo = O que o sensor leu REALMENTE - O que eu esperava ler
    float residuo = z_meas - z_pred;

    // Novo Estado = Predicao + (Ganho * "Surpresa do Sensor")
    f->estado[0] = x_pred[0] + K[0] * residuo;
    f->estado[1] = x_pred[1] + K[1] * residuo;

    // ATUALIZA A INCERTEZA (Covariancia Final)
    f->P[0][0] = P_pred[0][0] - (K[0] * P_zz * K[0]);
    f->P[0][1] = P_pred[0][1] - (K[0] * P_zz * K[1]);
    f->P[1][0] = P_pred[1][0] - (K[1] * P_zz * K[0]);
    f->P[1][1] = P_pred[1][1] - (K[1] * P_zz * K[1]);

    if (vel_out) *vel_out = f->estado[1];
```

```
    return f->estado[0];
}
```

## 5.4 Nota de Engenharia: A Armadilha do Excesso de Processamento

**Nota de Arquitetura: Por que não usar o UKF apenas para altitude?** Ao observar o código acima, vemos uma rotina complexa de fatoração de matrizes (Cholesky), loops calculando médias ponderadas e pontos espaciais espalhados. Tudo isso demanda considerável poder de processamento (ciclos de CPU).

O UKF brilha e é estritamente necessário quando as matrizes de transição ($F$) ou de observação ($H$) não formam retas (ex: funções com $\sin(\theta)$, $\cos(\theta)$, atrito aerodinâmico que depende de $v^2$). Nesses casos, a física "deforma" a curva da probabilidade, e a matemática pura do Linear Kalman Filter (LKF) quebra.

No entanto, no caso restrito de um altímetro onde estimamos apenas Posição ($p$) e Velocidade ($v$), a equação cinemática básica é $S = S_0 + V \times t$. Isso é **rigorosamente linear**. Se você propagar os Pontos Sigma do UKF por uma equação linear, os pontos não se deformarão; eles se moverão e se reconstruirão gerando um resultado matematicamente **idêntico** à multiplicação direta de matrizes do LKF.

Conclusão: Em sistemas 1D de navegação vertical simples, usar o UKF é um desperdício de bateria e tempo de processamento do microcontrolador. O LKF entrega a exata mesma precisão custando apenas uma fração das operações matemáticas da CPU. Reserve o UKF para a matriz do IMU (giroscópios e acelerômetros calculando atitude em Quaternions), onde a não-linearidade é a regra absoluta do sistema.